

WEEK 1 REPORT

Z24 Progressive Damage Test Data Preparation

Prepared by

TRAN VAN PHUONG

Reporting date

13 September 2026

Research target

Input X: sensor time series

Output y: bridge-condition classification label

This week: archive inspection, MATLAB reading, cleaning, segmentation, CSV export, metadata validation, and chart inspection.

1. Input data and complete flow

Input

The source is `raw_data/sources-Z24-004.zip`. The cleaner selects `pdt_01-08.zip` and `pdt_09_17.zip`. Each package contains conditions 01--17, avt and fvt measurement folders, setup folders, and MATLAB recordings.

MATLAB structure

`data` is a two-dimensional sensor time-series matrix. `labelshulp` stores channel names. At 100 Hz, adjacent samples are 0.01 seconds apart. A matrix of shape (65530, 33) contains 65,530 time samples and 33 channels.

Processing flow

Outer ZIP -> inventory headers -> ArchiveSlice for one nested ZIP -> ZipFile -> .mat payload -> `_read_mat` -> finite-row cleaning -> condition/setup parsing -> segments of at most 6,000 rows -> CSV and metadata.

Labels

The condition directory is the label source. condition 01 becomes label 0 and condition 17 becomes label 16. Labels are not inferred from sensor values.

All samples

`target_samples=None` keeps every finite sample. A 65,530-sample recording becomes ten full 6,000-row CSV files and one final remainder file of approximately 5,530 rows.

2. Function documentation: ZIP and path helpers

inventory(path)

Target: calculate the byte range of every ZIP entry. Mock input: `inventory("sources-Z24-004.zip")`. Flow: read local headers, names, packed sizes, uncompressed sizes, and completeness. Mock output: a list containing `C11-Appendices.pdf`, `pdt_01-08.zip`, and `pdt_09_17.zip` dictionaries. End byte = start + packed.

ArchiveSlice(path, start, size)

Target: expose one nested ZIP range as a virtual file. Mock input: `ArchiveSlice("sources-Z24-004.zip", 1060, 5000)`. Flow: map virtual position 0 to outer byte 1060 and limit reads to 5,000 bytes. Mock output: a stream whose first bytes may be `b"PK\x03\x04"`, passed to `zipfile.ZipFile`.

_outer_member(path, name)

Target: select one exact inventory record. Mock input: `path="sources-Z24-004.zip"`, `name="pdt_01-08.zip"`. Flow: iterate inventory and compare names. Mock output: the matching dictionary; missing names raise `FileNotFoundError`.

_condition_setup(path)

Target: parse metadata without opening the file. Mock input: `"01/avt/01setup03.mat"`. Flow: find condition 01, extract setup 03, detect avt. Mock output: `(1, 3, "avt")`.

3. Function documentation: MATLAB, CSV, and pipeline

`_read_mat(payload)`

Target: decode one .mat payload. Mock input: `payload = package.read(item)`. Flow: load bytes, require data and labelshulp, enforce a 2-D matrix, transpose if orientation is reversed, return float64 data and channel names. Mock output: data shape (3, 3) and names [sensor_A, sensor_B, sensor_C].

`_write_segment(path, values)`

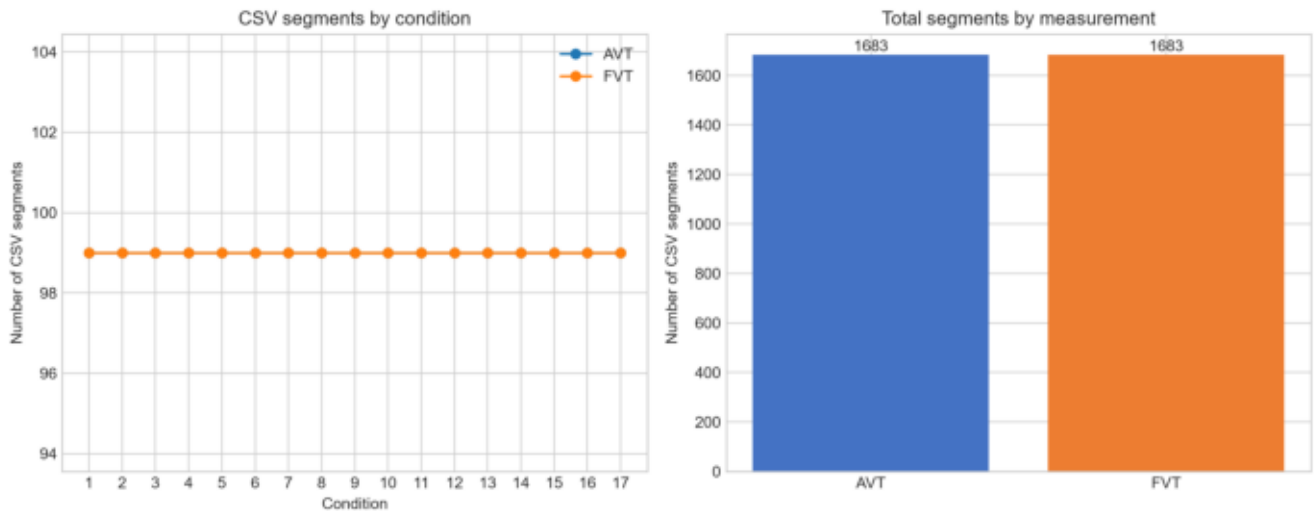
Target: create one CSV. Mock input: `values=[[0.1,0.2,0.3],[0.4,0.5,0.6]]`. Flow: create the header, prepend `sample_index`, and write one row per sample. Mock output: `sample_index,sensor_0,sensor_1,sensor_2` followed by data rows.

`clean_pdt(...)`

Target: coordinate the complete operation. Mock input: `archive=sources-Z24-004.zip`, `measurement=avt`, `max_conditions=None`, `max_setups=None`, `target_samples=None`, `segment_samples=6000`. Flow: read PDT members, parse paths, remove non-finite rows, keep all finite rows, write segments, and write manifest, labels, report, and status files.

Mock report output

```
{measurement: avt, recordings: 153, segments: 1683, csv_files: 1683, conditions: 1..17, issues: [], status: complete}
```

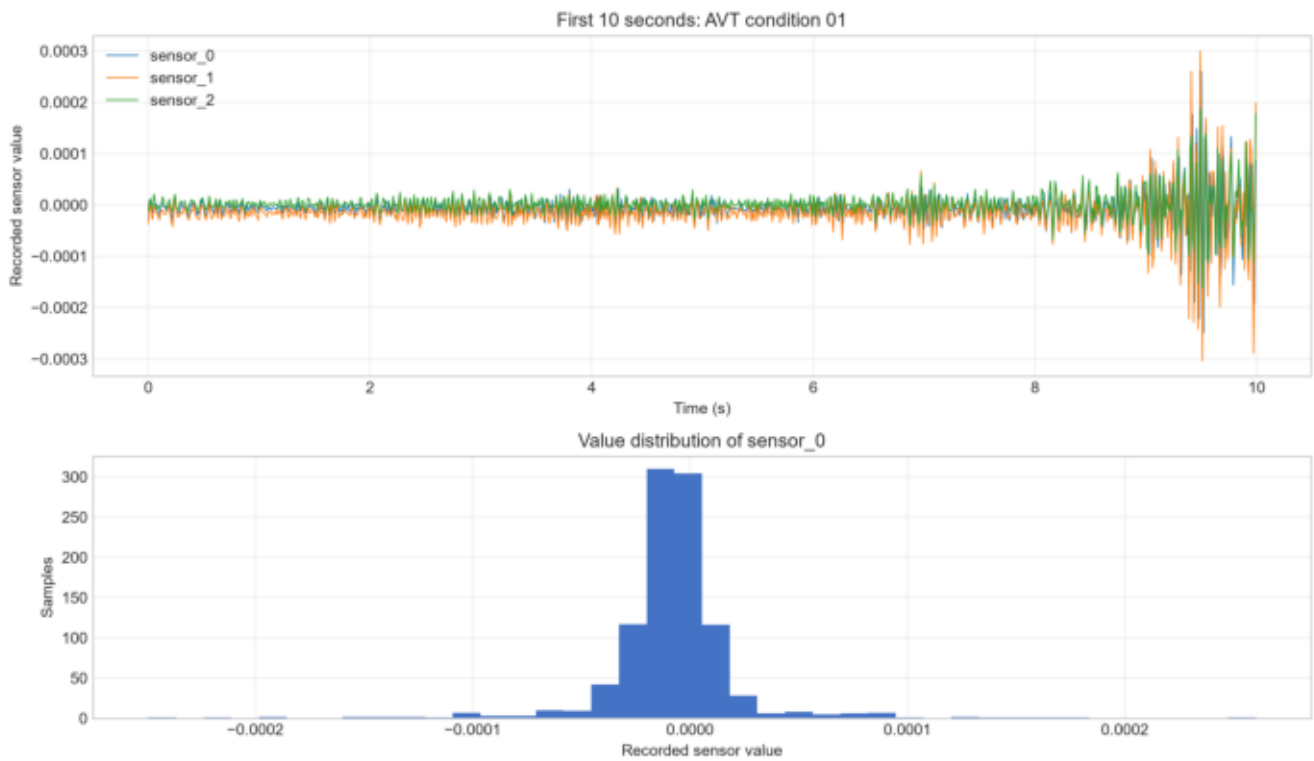


4. Clean CSV results and balance

The completed AVT run is stored in processed/avt/avt_13-09-2026_21-52-09. The completed FVT run is stored in processed/fvt/fvt_13-09-2026_21-54-21. Each run contains 153 recordings and 1,683 CSV segments, with no reported issues.

The chart shows 99 segments for every condition in both AVT and FVT, and 1,683 segments for each measurement type. This means the exported segment count is balanced across all 17 conditions.

The segments are not fully independent observations: segments from one recording are correlated. Train, validation, and test partitions must therefore be separated by recording or setup.



5. Signal interpretation

The three AVT channels oscillate around zero, which is consistent with vibration measurements. Their amplitude becomes larger near 8--10 seconds, with several short transients.

A vehicle crossing or another dynamic load could explain the stronger response, but the chart alone cannot confirm the event without a synchronized acquisition log. It also cannot classify structural damage by itself.

The sensor_0 histogram is concentrated near zero with small tails. Most samples have low amplitude and a smaller number represent stronger transients. No missing or infinite values are visible in this plotted segment. Sensor units must be confirmed from the PDT documentation.

6. Conclusion and next steps

Conclusion

The pipeline converts nested PDT MATLAB recordings into labeled, traceable CSV time series. The function-level behavior is documented with explicit targets, mock inputs, processing flows, and outputs.

Current output

AVT and FVT are stored in separate timestamped folders. Each output contains segments, manifest.json, labels.csv, report.json, and status.json. All finite samples are retained; the final segment may be shorter than 6,000 rows.

Next steps

Confirm sensor units and setup consistency; choose whether AVT and FVT will be modeled separately; split by recording or setup to prevent leakage; normalize from training data only; handle the shorter final segment; and create fixed-length tensors for the classifier.

Training meaning

The cleaned data provide the required supervised structure: X is the sensor time series and y is the condition label. Model training and evaluation are the next phase.